DESIGN DOCUMENT

MTI Radio Management

System design: architecture, modules, data model, register rules, integrations, security, deployment, testing and releases.

Application version: 1.21.4 · **Date:** 21 October 2026

For: the IT team and developers who maintain, operate or extend the application.

Related documents in `docs/`: User Guide and API Documentation. Full technical detail is in `README.md`.

Contents

1. Introduction 3

2. Architecture 5

3. Application modules 7

4. Data model 9

5. Register rules 12

6. Integrations 14

7. Security 15

8. Main flows 17

9. Deployment and operations 20

10. Testing 22

11. Versions and releases 23

12. Design decisions and open points 24

1. Introduction

MTI Radio Management replaces the "Form" sheet of the old radio register workbook with a web application: every MOTOTRBO handheld (HT) and rig radio owned or used by MTI is recorded once, with its holder, condition and history.

Goals

- One shared source of register data, replacing Excel copies that differ from each other.
- Rules the system enforces: a unique Radio ID and SN, no letter F in the MDC Code of a Radio ID, one save at a time.
- A complete trail: a history per radio and an Audit Log of every change and sign-in.
- Everyday register work is recorded in the application: check results become a work list, radios sent for service are tracked until they come back, and stock taking is done from a phone.
- Supporting documents in SharePoint (COF scans and the ICT Masterlist) can be viewed and cross-checked from the application.
- IT assets recorded in Snipe-IT can be viewed from the same application, without opening Snipe-IT and without changing it.
- Other applications can use the data through the API, with the same rules and permissions.
- The application can be used in Indonesian or English, as each account chooses.

Scope

Included	Content
Radio register	List, add, edit, archive, restore, delete, select many, Excel export, import of the old register, history, Radio ID picker
Master data	Radio types, companies, departments, channel templates
Handover	Handover dialog with the recipient's signature (for printing only), the IT Equipment Checkout form filled from the register, the ICT officer's signature (E-Sign)
Needs Attention	Register check results as a work list: fixes per kind of check, dismissal with a reason, assignment
Service & Repair	Service tickets per radio from reported to returned, service time and cost, Excel export
Stock opname	Phone page: a session per department, scanning the SN barcode, checking holder and condition, sending the result to the register
SharePoint documents	COF Scan folder (with linked / not linked counts), ICT Masterlist workbook, cell change requests and approvals, direct row edits by approvers
IT assets (Snipe-IT)	Snipe-IT's content, read only: overview, assets with pictures and their details page, catalog, people and organization, licenses and stock, history
Access and trail	Microsoft, LDAP and password sign-in; User Management, Role & Permission; Audit Log; notifications
Language	Indonesian (default) and English on every page, chosen per account; stored data stays in Indonesian
Integration and operations	API `/api/v1` with tokens; health checks `/up` and `/api/v1/health`; version history in the footer

Out of scope: programming the radios themselves (codeplugs), e-mail notifications, changing data in Snipe-IT (the application only reads), and the template's Approval Management module (hidden).

Terms

Term	Meaning
Radio ID (DMR ID)	The radio's number in the radio system, unique among active radios (`dmr_id`)
MDC Code	The Radio ID in upper-case hexadecimal without leading zeros (1001 → 3E9), calculated automatically
SN	Serial number; one register row = one physical radio
Location, Condition	Where the radio is now (With user, IT stock, In service, Lost), and its state (Good, Needs service, Broken, Not checked)
COF	IT Equipment Checkout Form (MTI-HRM-ICT-FRM-001)
Needs Attention item	One check result that is still open: one per radio, or one per SN / Radio ID used by several radios
Service ticket	One radio sent to a service centre, numbered `SRV-0001`, with the steps Reported → Sent for service → In repair → Ready for pickup → Returned
Opname session	One count of the radios of a department (and company), from its start until it is sent to the register
Menu, permission	One page in the sidebar, and the Read, Create, Update and Delete permissions on that page per role
Snipe-IT	A separate IT asset inventory system; this application reads its content through the Snipe-IT REST API
Health check	An address that monitors and Docker call to know whether the application works normally

2. Architecture

One Laravel web application serves both the pages and the API. The application database and every other service are outside the container; the application contacts each of them for one purpose.

One application in the middle; the database and other services are outside the container

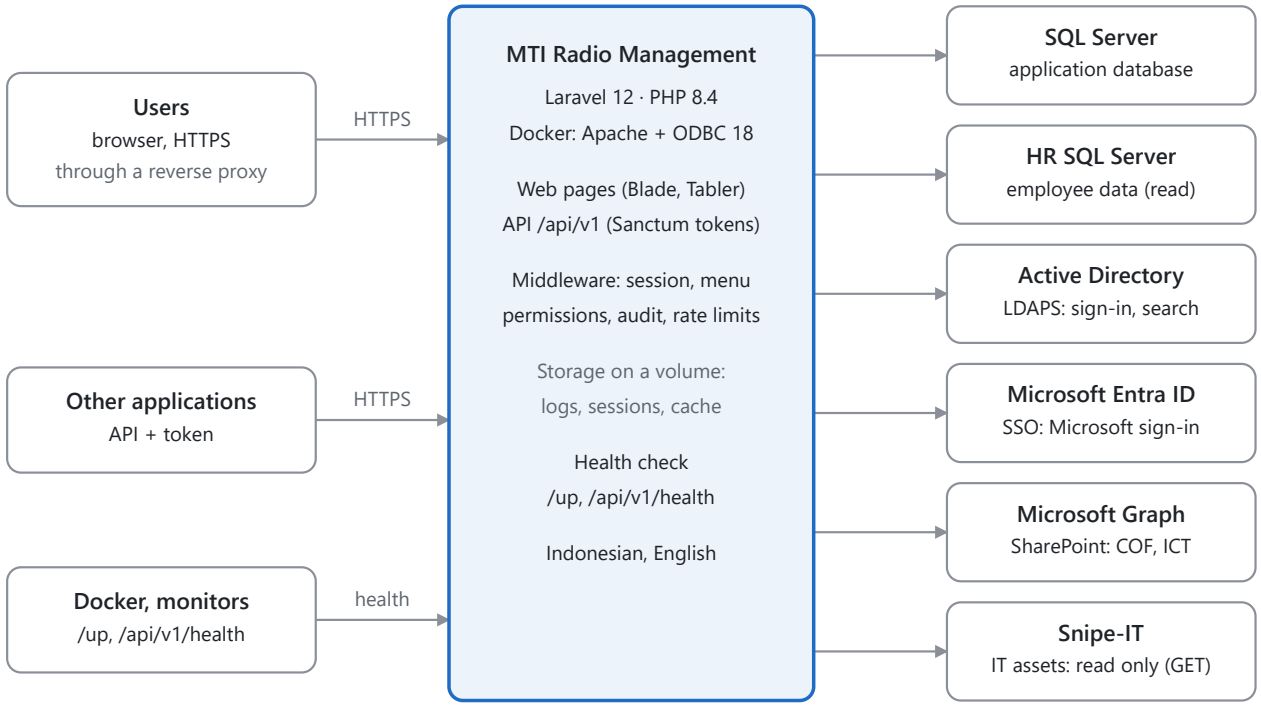


Figure 1. System context.

Technology

Layer	Choice
Language and framework	PHP 8.4, Laravel 12, built from the internal Base-Apps-Merdeka template (modules in `modules/`, menu/role/permission RBAC)
User interface	Blade, the Tabler theme (Bootstrap 5) with a design layer of its own, `public/assets/css/radio-ui.css` (colour tokens `--rm-*`, components, motion), jQuery, SweetAlert2, Tabler Icons; no front-end build
Camera	The browser's built-in `BarcodeDetector` to scan SN barcodes in Stock Opname (Chrome on Android); no extra library
Database	Microsoft SQL Server through the `sqlsrv` driver (ODBC 18)
Sign-in	Laravel Socialite with the Microsoft Azure provider (SSO), FreeDSx LDAP (pure PHP, without the `ldap` extension)
API	Laravel Sanctum (tokens), Scramble (automatic OpenAPI documentation at `/docs/api`)
Excel	Maatwebsite Excel (export), its own `.xlsx` reader and writer for the SharePoint workbook
Languages	Laravel language files: `lang/id` and `lang/en` for the shared parts, `Resources/lang/<code>` in each module

Layer	Choice
Snipe-IT	Laravel's HTTP client (GET only), its answers kept for a while in Laravel's cache
Monitoring	Health checks `/up` and `/api/v1/health`, Laravel logs, the `activity_logs` and `error_logs` tables, Sentry (optional)
Container	Docker: `php:8.4-apache` with the SQL Server driver, one application container

3. Application modules

Features are split into modules under `modules/`; services used by many modules are in `app/`. Business rules live in services, so the pages and the API run the same code.

Module	Responsibility	Key parts
`RadioManagement`	Radio register, history, master data, Excel import and export, checkout form, handover, select many, E-Sign Master, Needs Attention, Service & Repair, Stock Opname	`RadioService`, `RadioRules`, `RadioIdService`, `RadioCheckService`, `RegisterLock`, `RadioImportService`, `RadioBulk`, `CheckoutForm`, `ESigns`, `AttentionInbox`, `ServiceTickets`, `RadioOpnames`
`Dashboard`	Register summary: numbers, condition of active radios (pie), active radios per user company (donut) and per type (bars), check results, free IDs, activity	`RadioSummary`, `Pie`
`SharePointDocs`	COF Scan, ICT Masterlist, cross-check, cell change requests and approvals, direct row edits	`GraphClient`, `SharePointFolder`, `FolderView`, `SharePointWorkbook`, `WorkbookReader`, `XlsxPatcher`, `ScanRegister`, `CellEdits`
`UserManagement`	Accounts (sign-in method, employee search, status, password), Role & Permission	`UserScope` (reach per company), `EmployeeDirectory`, the LDAP directory
`SnipeIt`	The Snipe-IT menu (IT Assets section): overview, lists per group (assets, catalog, people and organization, licenses and stock, history), details of one asset; read only	`SnipeItClient` (GET only, the token stays on the server), `Answers` (cache), `Lists`, `Rows`, `Summary`, `AssetDetail`, `AssetPage`, `Names`
`AuditLog`	The audit trail page (table and timeline, filters by kind of record, by action with readable names, and by date range)	`AuditLogQuery`, `UserAgent`
`Notifications`	The notification bell and the Notifications page with a tab per category (Radio, Documents, System)	`Notifier`, `NotificationSweep`
`Api`	API `/api/v1`, health checks, the API Tokens page, API documentation	Middleware `ApiAccess`, `PersonOnly`, `LimitAddresses`; `ApiTokens`; `HealthController`, `Health`
`ApprovalManagement`, `Sweetalert`	From the template; the Approval Management menu is hidden and not used	–

Other template leftovers that were never used were removed in version 1.20.1: every `x-global` Blade component except `btn-detail` and `summary-card`, the tinymce, bootstrap and list.js libraries, and the mSafe and orangeHR models with their database connections.

Shared parts in `app/`

Part	Purpose
`Auth/LoginController`, `Directory/LdapAuthenticator`	Microsoft, LDAP and password sign-in; refusals are recorded in the Audit Log
`CheckMenuPermission`	Menu permission per request: route name → menu, method → permission
`SignOutInactive`, `EndExpiredSession`, `SessionClock`	End the sessions of deactivated accounts, and sessions past 30 minutes without activity or 8 hours since sign-in
`LogActivityMiddleware`	Records requests that change data in `activity_logs` (sensitive fields masked)
`Audit/AuditLogger`	Writes the audit trail with the values before and after
`Notifications/Notifier`, `NotificationSweep`	Write notifications; time-based notifications are made when the person opens the application
`Audit/UserAgent`	Names the browser and system from the stored user agent ("Chrome 141 on Windows")
`Language`, `SetLocale`, `LanguageController`	The language of each request: the account's choice, then the `locale` cookie, then Indonesian; `POST /language` saves the choice (section 8)
`Breadcrumbs`, `Menu::label()`	The path below the top bar, built from the menu tree ("Dashboard › Register Radio › Radio 1053"), and menu names in the page's language
`Changelog`, `ChangelogController`	Version history in the footer: reads `CHANGELOG.md` or `CHANGELOG.en.md` into versions, dates and notes per kind of change (section 11)
`public/assets/css/radio-ui.css`, `main/components/js`	The design layer and the shared page scripts: result messages (`rmFeedback`), confirmation dialogs (`rmDialog`), date and multi-choice pickers (`rmPickers`), motion (`rmMotion`), tables (`renderTableWithFeatures`)

4. Data model

One `radios` row is one physical radio. A radio points to four master tables; its history and the uses of signatures point back to the radio. Radios and accounts are never deleted permanently (soft delete).

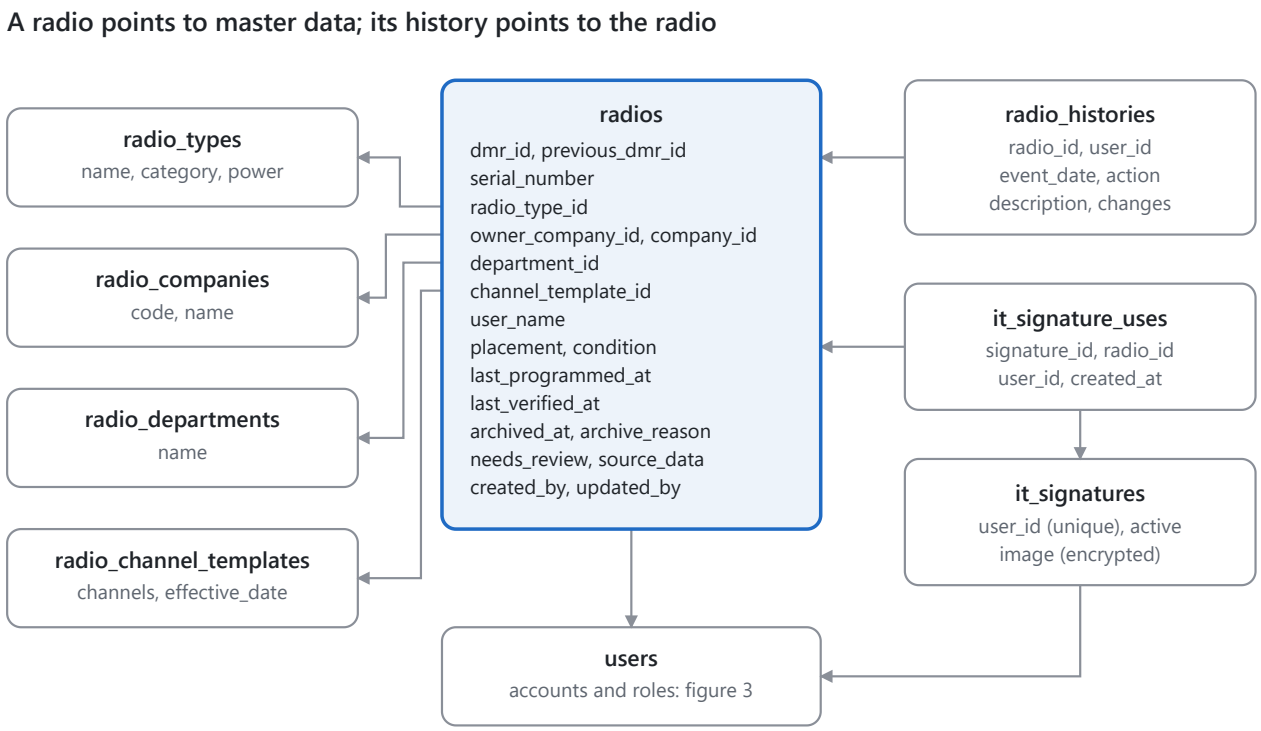


Figure 2. Register tables. Arrows point from the foreign key column to the table it refers to.

Main tables

Table	Content and design notes
`radios`	One physical radio. `placement` and `condition` are stored as codes (enums in the code). `dmr_id` is empty while the radio is archived, its old ID kept in `previous_dmr_id`. `source_row`, `source_sheet` and `source_data` keep the original Excel row on import; `needs_review` and `review_notes` mark Excel data that contradicts itself. The MDC Code and the Checks column are calculated, not stored
`radio_histories`	One event on one radio: `action` (stored in Indonesian: Serah Terima, Ubah Status, Program, Verifikasi, Arsip and others; shown with its name in the page's language, such as Handover or Status changed), `event_date`, `changes` (JSON per field: label, old, new), `user_id` (empty for the system)
`radio_types`, `radio_companies`, `radio_departments`, `radio_channel_templates`	Master data with `is_active`; a unique name or code. Inactive entries stay on older radios but cannot be chosen; only unused entries can be deleted
`users`	Accounts of people and API clients (`api_client`). `ldap_username` (Directory username), `microsoft_linked_at`, `last_login_at`, `last_login_method`, `active`, `locale` (the account's language, `id` or `en`; empty means not chosen yet), soft delete

Table	Content and design notes
`roles`, `role_user`, `menu`, `menu_role`, `permissions`, `menu_role_permission`	The template's RBAC (figure 3)
`audit_logs`	Audit trail: actor, action, the record touched, values before and after (masked), IP, user agent, `via` (the API and its token). Never changed or deleted by the application
`activity_logs`, `error_logs`	Log of requests that change data, and error log (from the template)
`sharepoint_edit_requests`	One cell change request: the cell, the row and its identifying cell, old and new value, status, requester, approver, result of the write. `direct` marks cells written directly through a row edit
`it_signatures`, `it_signature_uses`	ICT officers' signatures (one per account, the picture encrypted with `APP_KEY`) and a record of every use on a form
`personal_access_tokens`	Sanctum API tokens: the token hash, read/write ability, expiry, when and from which IP it was last used
`user_notifications`	Notifications per account with a `dedupe_key` unique per account, so that one thing is only notified once

Two migrations came after 1.10.0: `2026_10_21_100000_add_locale_to_users` (the `users.locale` column, version 1.13.0) and `2026_10_21_110000_add_snipe_it_menu` (the Snipe-IT menu group in the IT Assets section with its six pages, given to Super Admin with the Read permission, version 1.15.0). Before the `locale` column exists, the language choice is kept in the browser only and nothing fails. Snipe-IT data is not stored in the application database: its answers are only kept for a while in the cache.

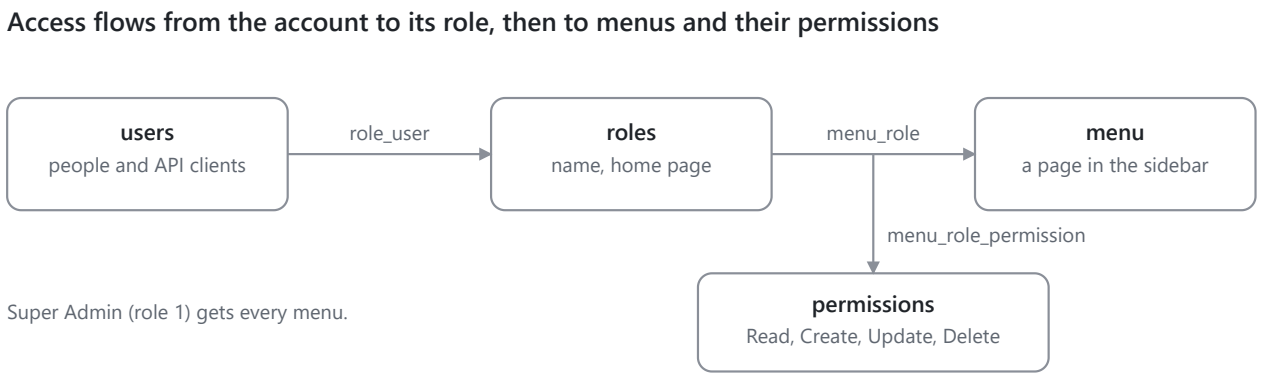


Figure 3. Access tables. Permissions are stored per pair of role and menu.

Needs Attention, Service and Stock Opname tables

These three abilities refer to radios without a foreign key: an Excel import with "delete the old data" deletes every radio, so their rows copy the radio's label (Radio ID, SN, type) when they are made and stay readable afterwards. While a ticket or an opname session is open, such an import is refused.

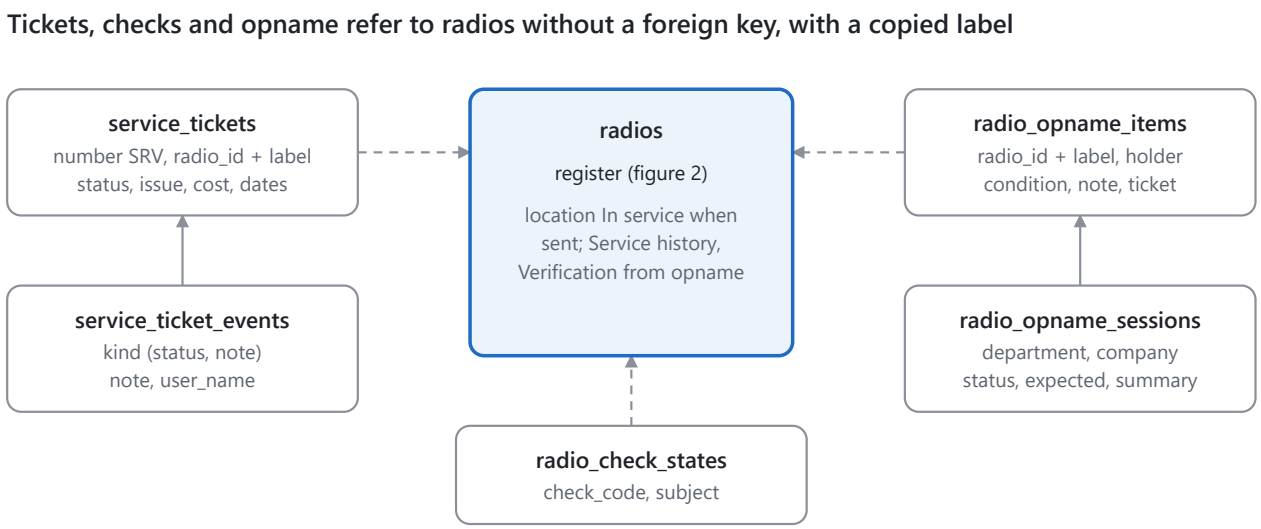


Figure 4. Service & Repair, Needs Attention and Stock Opname tables.

Table	Content and design notes
`service_tickets`	One service ticket: the number `SRV-0001`, the radio and its label, the issue, reporter, service centre, BA No., date sent, expected return, date returned, quote, cost, status, the radio's location before it was sent. One open ticket per radio
`service_ticket_events`	Ticket activity: every step and every note, with the name of its writer
`radio_check_states`	One Needs Attention item per kind of check and subject (radio, SN or Radio ID): when it was first seen, whom it is assigned to, why it was dismissed. A fingerprint of the data at dismissal makes the item come back when the data changes. The check results themselves are still calculated, not stored
`radio_opname_sessions`	One opname session per department (and company): status open or sent, the number of radios expected, a summary when it is sent
`radio_opname_items`	One radio scanned in a session: who holds it (as in the register, someone else, nobody), condition, note, the ticket number when it was reported with a problem

5. Register rules

The register rules live in services (`RadioService`, `RadioRules`, `RadioIdService`, `RadioConflicts`) and are shared by the pages, select many, the import and the API.

Rule	Design
Radio ID	1–16776415, unique among active radios. An ID whose MDC Code contains the letter F is refused (F is the wildcard character in MDC-1200). The last ID of an archived radio stays reserved for when it is restored. The Radio ID picker in the form (`GET radio/id-check`) answers while typing: whether the ID can be used, and if not why and who holds it, a replacement ID, and the IDs around it
SN	At most 50 characters, unique
Location and Condition	The old Excel Status column split into two dimensions; the value "Available" from old audits is imported as Good
Owner	MTI or Non MTI, calculated from the company code of the Asset owner, not stored
Checks column	Calculated when read (`RadioCheckService`): duplicate SN/ID, ID containing F, empty SN/ID/type, SN format, lost without a BA, needs reprogramming (programmed before the effective date of the template), verification empty or older than 12 months, Needs checking. The results are worked on in Needs Attention (section 8)
Service	One open ticket per radio; archived radios cannot be serviced. When a radio is sent, its Location becomes In service; when it returns, its Location (With user or IT stock) and Condition are chosen. Both write a "Service" history row with the ticket number in its description and the BA No. as document number (`RadioService::serviceStep`)
Stock opname	Sending a session to the register writes a "Verification" history row for every radio found that day, and updates conditions that changed
Handover	A dialog on the radio details: the recipient becomes the holder, a "Handover" history row with the BA No. as document number, then the checkout form opens with the recipient's signature
Archive	A radio that leaves for good is archived (Moved to another site, Written off, Lost (BA signed), Other): `dmr_id` is released into `previous_dmr_id`. When it is restored, the old ID comes back if it is still free
Delete	Only for wrong entries; soft delete, the history stays
Automatic history	Every save is compared with the old data, then classified: a change of holder, company or department becomes Handover; Location or Condition becomes Status changed; the programming date becomes Programming; the verification date becomes Verification; anything else Data changed

Concurrent saves

- **One save at a time** (`RegisterLock`, a cache lock): two people saving at the same moment are handled one after the other, so a Radio ID is never used twice.
- **Radio version** (`version`, from the time of the last change): the edit form and the API send the version they read; if someone else has changed the radio since, the save is refused ("just changed", HTTP 409).
- **An import** holds the register lock while it runs; other saves get "The register is busy". The lock of an interrupted import is released by itself after at most 10 minutes, or with `php artisan radio:unlock`.

Importing the old register

One Excel row with an SN or a User Name becomes one radio; empty slot rows are skipped. The spelling and content of the Status column are cleaned up into Location and Condition. Contradicting data (a duplicate SN or ID, a status that contradicts the notes) is not guessed: the radio is still imported and marked Needs checking. Values that do not fit their column are cut or left empty, also with the Needs checking mark. The import only fills an empty register; replacing a register that holds data needs the "delete the old data" option and the Delete permission, and is refused while a service ticket or a stock opname session is open (`RadioImportService::refuseWhileOpen`).

6. Integrations

Every integration is optional per environment: when its settings are empty, the feature is not shown or gives a message, and the rest of the application keeps working.

System	For	Design
Microsoft Entra ID	Microsoft sign-in (SSO)	OAuth 2.0 / OpenID Connect through Socialite. The e-mail matched is the `userPrincipalName`; the account must already be registered, active, and have a role. The account choice is always shown. The callback address must be the address users open, registered in the Entra app
Active Directory	LDAP sign-in, finding accounts when adding a user	FreeDSx LDAP, always encrypted (LDAPS, or StartTLS before a password is sent). The password is checked by binding as that person; no password is stored. Searches use a service account and only offer active personal accounts. The server certificate is verified with the internal CA (`LDAP_TLS_VERIFY`, `LDAP_CA_CERT`), so its address is written as a DNS name, not an IP
HR SQL Server	Finding employees for Microsoft accounts, and handover recipients	A read-only connection to three columns (number, name, work e-mail), preferably through a view; only company e-mail domains are offered
Microsoft Graph (SharePoint)	COF Scan, ICT Masterlist, cell changes	The application's own identity (client credentials), the application permission `Sites.Read.All` to read and write permission on the site for cell changes. Folder lists and workbook content are cached (10 minutes by default) and still shown while Microsoft cannot be reached. The Excel REST API is not available to an application identity, so a cell change is written by downloading the `.xlsx`, changing one cell in its XML (`XlsxPatcher`), then saving it with `If-Match` so that clashing writes are refused. A direct row edit does the same for several cells at once: one download, every cell checked, one upload
Snipe-IT	The Snipe-IT menu: viewing IT assets and their supporting data	The Snipe-IT 6.2 REST API v1 with a token from `.env`, GET only (`SnipeItClient`). Answers are cached for `SNIPEIT_CACHE_MINUTES` minutes (5 by default). When Snipe-IT cannot be reached, the application stops asking for 60 seconds and the page gives a message. API traits that shape the design: texts come HTML-escaped (they are unescaped before they are shown); `fields` and `fieldsets` ignore limit, offset and search; an asset that does not exist is answered with 200 and the status `error`; an asset's history comes from `reports/activity` (with `item_type` and `item_id`), because the components, files and history of an asset are not available through the API (404). Components and files link to the Snipe-IT page. Pictures are loaded by the browser straight from Snipe-IT's address
Other applications	Reading and changing the register	API `/api/v1` with Sanctum tokens; the same rules and permissions as the pages (see the API Documentation)

> The COF Scan and ICT Masterlist pages read SharePoint with the application's identity. So who may see their content is decided by the menus in the application, not by the person's own access in SharePoint. Every file shown or downloaded is recorded.

7. Security

Authentication

- Three ways in: Microsoft (SSO), LDAP (the Active Directory password), and a local password set by an admin. Accounts are made by admins; there is no self-registration and no password reset.
- The password form is limited to 10 attempts per minute per address; refusals are recorded in the Audit Log with their reason.
- Accounts without a role and deactivated accounts are refused by every way in; a Microsoft sign-in with a public e-mail address is refused too.

Sessions

- Stored as files on the server (`SESSION_DRIVER=file`); the cookie is HttpOnly, SameSite=Lax, and Secure in production. The session ID changes at sign-in; signing out deletes the session.
- Two limits on the server (OWASP, NIST SP 800-63B-4): 30 minutes without activity and 8 hours since sign-in. Pages warn 2 and 5 minutes before. The page's automatic requests (the bell, the session check) do not count as activity.
- No "remember me"; old remember cookies do not revive a session.
- A deactivated account is signed out on its next request.

Layers of every page request

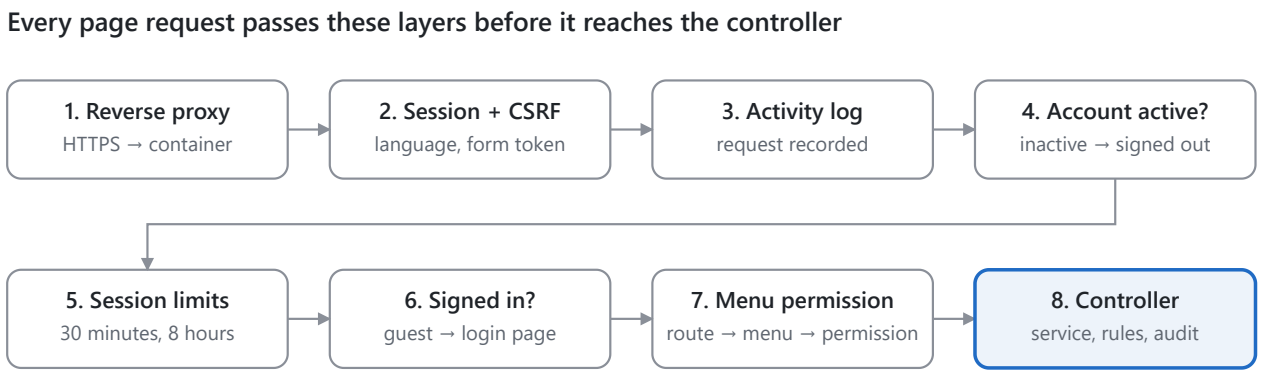


Figure 5. Page request layers (the web group and route middleware).

The language is decided twice: in the global middleware from the cookie only (so that the 404 and maintenance pages speak it too), then right after the session starts, when the account's choice wins (`SetLocale`). `/up` passes none of these layers: no session, database or token.

Authorization

- `CheckMenuPermission` takes the menu from the route name (without its last part) and the permission from the method: GET Read, POST Create, PUT/PATCH Update, DELETE Delete. Super Admin skips this check.
- Buttons that may not be used are not shown (`@permission` in Blade), but the server checks every request again.
- User Management is limited per company for restricted role types (`UserScope`); Super Admin can only be managed by Super Admin; nobody can delete or deactivate their own account.
- Cell change approvals: a requester never approves their own request. Direct row edits are only for personal accounts (not API clients) with Update on the ICT Masterlist and on Edit Approvals.

- Pages whose menu is not yet in the database are closed to everyone but Super Admin (`RequireMenu`). Fixing a Needs Attention item needs Update on that menu and on Register Radio; reporting a problem from Stock Opname needs Create on Service & Repair.

Trail and data protection

- **The Audit Log** records sign-ins (successful, failed with the reason, sign-outs, ended sessions) and every data change with the values before and after; passwords, tokens and signature pictures are never written.
- **Signature pictures** of ICT officers are stored encrypted with `APP_KEY`, redrawn by the server as PNG, and never leave through the API. The recipient's signature (in the handover dialog or on the form) is not stored at all, not in the Audit Log either. Changing `APP_KEY` makes stored signatures unreadable.
- **Secrets** are only in `.env`, which is in neither the repository nor the Docker image. Files with real data (the register workbook, Word forms) are ignored by git.
- **Proxy headers** are only trusted from `TRUSTED_PROXIES`, so the visitor's real address is recorded and used for the rate limits. An `APP_URL` starting with https makes every link https.
- **Snipe-IT** is only read: no route sends anything but GET to Snipe-IT. The Snipe-IT token never leaves the server (not in a page, an answer or the log). Only the columns shown in a list reach the browser; anything else in Snipe-IT's answers (for example license keys, people's phone numbers and addresses) is never sent. Purchase cost and book value only show on the asset details page. Custom fields whose name marks a secret (password, PIN, BitLocker, recovery, key, token) do not show their value, nor do password values in the history. The token should belong to a Snipe-IT account that may only view.
- **The health check** without a token only gives the status of each essential check. Details per service are only for a valid API token (any token, not a menu permission). Its answers never name a server, a path or an error message of a server; the reason is in the log.
- **The language cookie** (`locale`) is deliberately not encrypted: it only holds a language code, and any value other than `id` or `en` is ignored.

API

- Sanctum tokens, stored as a hash, prefixed `mtiradio_`, with a read or a read & write ability, an expiry, and revocable. A browser session does not open the API.
- API layers: answers are always JSON, a limit per address (120/minute) before the token is checked, the API is active (its migrations have run), the token's account is active, the menu permission matches the method and the token's ability, then a limit per token (60/minute; import/export 10, SharePoint 30).
- CORS is closed except for the sites listed in `CORS_ALLOWED_ORIGINS`.
- API messages are in English; clients that send `Accept-Language: id` get Indonesian. Data and labels in the answers stay as stored.
- `/api/v1/health` skips the usual API checks (API active, token required), so that it still answers when the database is down; the limit per address (120/minute) still applies.

8. Main flows

Handover and the checkout form

1. On the radio details, the officer opens the Handover dialog: the recipient (looked up in the employee directory or typed), BA No. (required), PO No., accessories, condition, the location afterwards, the recipient's department and company, and the recipient's signature on a canvas (pen, finger, mouse; pen pressure through Windows Ink, the palm ignored).
2. `RadioService` saves the recipient as the holder and records a Handover history row (the BA No. as document number) and the Audit Log entry `radio.handed_over`.
3. The checkout form opens in a frame of its own, filled from the register (`CheckoutForm`), with the recipient's signature in the RESPONSIBLE USER box. Fields can be changed on the form for that printout only; the signature only exists on that page. The form can also be opened without a handover (the Print button) and signed there.
4. If the officer's account has a signature in E-Sign Master, it appears in PROCESSED BY and its use is recorded in `it_signature_uses`.
5. The form prints on one A4 page (scaled down automatically when needed) or is saved as PDF.

ICT Masterlist cell changes

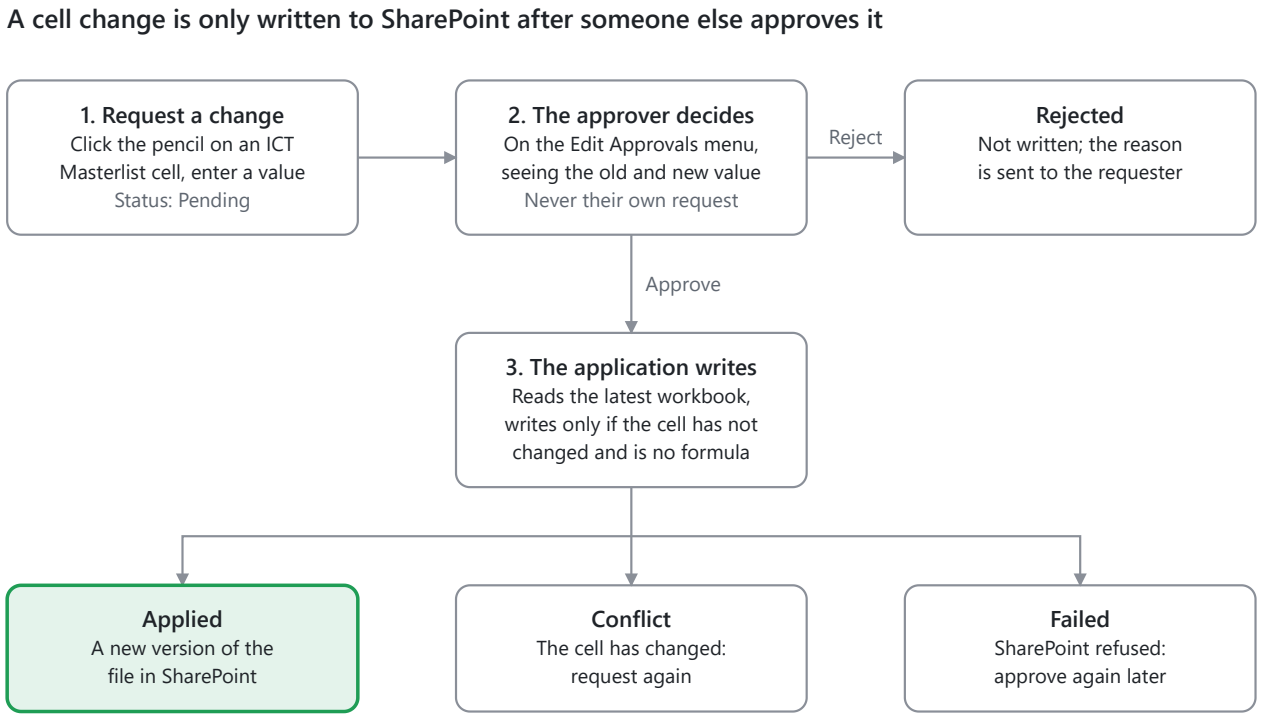


Figure 6. The cell change flow (`CellEdits`, table `sharepoint_edit_requests`).

A request keeps the cell's old value and the row's identifying cell (for example the COF No). On approval the workbook is read again, and the cell is only written if both are still the same, the cell is not a formula, and nobody else saved the file in between (`If-Match`). Every write becomes a new version of the file in SharePoint. Requests, decisions and their results are recorded in the Audit Log and trigger notifications.

Direct row edit (`CellEdits::writeRow`): an account that may both request and approve writes several cells of one row without anyone else's approval. The workbook is downloaded once; every cell and the row's identifying cell are checked as an approval would check them; all cells are changed and uploaded once with `If-Match`. Nothing is

written when the row has changed (409), someone else saved the file (412), or Excel holds a lock on it (423). Every cell is recorded as a request applied by its writer, marked `direct`, and in the Audit Log as `document.edited_directly`.

Needs Attention

`AttentionInbox` groups check results into items: one per radio, or one per SN / Radio ID used by several radios. When an item is first seen, its row in `radio_check_states` is made. Each kind of check offers its own fix (correct the SN, archive a duplicate, the next free ID, give back the last ID, choose a type, fill in the BA No., record programming or verification, mark as reviewed). Every fix runs through `RadioService`, so the history, the Audit Log and the register lock apply as usual. Dismissing needs a reason and keeps a fingerprint of the data; the item opens again when the fingerprint changes. Assignments and dismissals are recorded as `attention.assigned` and `attention.dismissed`. The number of open items shows in the sidebar.

Service and repair

Step	What is saved	Effect on the radio
Reported	The radio (label copied), the issue, the reporter; the next `SRV-0001` number	–
Sent for service	Service centre, date sent, BA No., expected return, quote; the radio's previous location	Location In service, a "Service" history row
In repair	Quote, expected return	–
Ready for pickup	Cost	–
Returned	Date returned, cost	The chosen location and condition, a "Service" history row

`ServiceTickets` only moves a ticket to its next step; if someone else moved it first, the request is refused. Every step and note becomes a `service_ticket_events` row and an Audit Log entry (`service.created`, `service.status_changed`, `service.note_added`). The page's numbers: open tickets, the average service time over the last 90 days, late tickets (more than 14 days), the cost this quarter.

Stock opname

1. A session is opened per department (and company); a session still open there is continued. `expected` is the number of active radios in that department when the session starts.
2. A radio is looked up by Radio ID or SN: a scanned barcode (`BarcodeDetector`), a handheld scanner, or typing. Every answer (holder, condition, note) is saved at once as `radio_opname_items`.
3. Report a problem makes a service ticket through `ServiceTickets`, and its number is kept on the item.
4. Send to the register writes a Verification history row for the radios found that day, updates the conditions that changed, keeps a summary, and records `radio.opname_submitted`.

Notifications

Event notifications (a new request, a decision) are written when the event happens. Time-based notifications (API tokens expiring within 7 days, the weekly "Needs action" summary) are made by `NotificationSweep` when the person opens the application, at most once an hour per person; the application uses no scheduler. `dedupe_key`, unique per account, prevents duplicate notifications, also when two installations share the database. Notifications are in-app only; read ones are deleted after 90 days.

Select many

Bulk actions in Register Radio and Users handle every record with the same rules as the single action (history, Audit Log, register lock), one by one. Records that cannot be handled are reported with their reason, while the rest are still handled. The limit is 100 ticked records, or 1,000 filtered radios for verification.

Page language

1. The language of a request (`Language::forRequest`): the account's choice (`users.locale`), then the `locale` cookie, then Indonesian. The login page uses the cookie, so it stays in the last language after signing out.
2. The ID | EN switch in the top bar (in the sidebar on phones, and below the login form) sends `POST /language`: the choice is saved on the account and in the cookie, then the same page opens again. A choice made in another browser reaches this browser on its next request.
3. Page texts come from the language files; texts for page scripts are passed through `@json(__(...))`. Numbers, dates and month names follow the page's language. Menu names come from `lang/<code>/menu.php`.
4. Data stays in Indonesian, because others use it and it is stored: radio history, the Audit Log, Excel import/export files, the printed checkout form, labels in the REST API. Locations, conditions, archive reasons, accessories and kinds of history are only shown with their name in the page's language. Notifications are written in their recipient's language.
5. Terms deliberately left untranslated: Dashboard, Register Radio, Stock Opname, COF Scan, ICT Masterlist, E-Sign Master, Import Excel, Audit Log, API Tokens, User Management, Super Admin, Role, and the kinds of release and change in the Version history.

The Snipe-IT menu

Every page (Overview, Assets, Catalog, People & Organization, Licenses & Stock, History) is a sidebar entry with its own permissions; opening a page needs access to its menu, loading its rows needs Read. A page loads its rows from the `list` route (JSON) in the application, never straight from Snipe-IT. `Lists` decides the endpoint, columns and order of every list; `Rows` shapes the cells (texts unescaped from HTML, dates, links to Snipe-IT, a picture or a category icon). The overview counts every list and the assets per status and category. The details of one asset (`snipe-it.assets.show`, needs Read on Assets) ask for several answers at once: the asset, its history, maintenance, licenses, and the assets checked out to it (`AssetDetail`); `AssetPage` then arranges them into the asset's data, purchase and warranty, custom fields, history and audit dates.

9. Deployment and operations

Environment	Form
Production	One Docker container (`php:8.4-apache` with the SQL Server driver) on a Docker server, behind an HTTPS reverse proxy (Nginx Proxy Manager). Host port 26102 (`DOCKER_PORT`). Storage (logs, sessions, cache) on the `radio-management-storage` volume; the CA certificate for LDAP is mounted from the `certs/` folder (read only). The container healthcheck calls `/up` every 60 seconds
Development	`php artisan serve` (port 8020) on the developer's laptop, opened through a local https name with Caddy as a local reverse proxy
Testing	A separate SQL Server LocalDB database; never the live database

Settings (`.env`)

Group	Keys
Application	`APP_URL`, `APP_KEY`, `APP_ENV`, `APP_DEBUG`, `APP_TIMEZONE` (Asia/Makassar, WITA)
Database	`DB_CONNECTION=sqlsrv`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`, `DB_ENCRYPT`
Microsoft	`MS_GRAPH_TENANT_ID`, `MS_GRAPH_CLIENT_ID`, `MS_GRAPH_CLIENT_SECRET`, `MS_SSO_REDIRECT_URI`
SharePoint	`SHAREPOINT_COF_SCAN_URL`, `SHAREPOINT_ICT_MASTERLIST_URL`, `SHAREPOINT_DOCS_CACHE_MINUTES`
Active Directory	`LDAP_URL`, `LDAP_BIND_DN`, `LDAP_BIND_PASSWORD`, `LDAP_USER_SEARCH_BASE`, `LDAP_PEOPLE_SEARCH_FILTER`, `LDAP_TLS_VERIFY`, `LDAP_CA_CERT`
HR database	`HR_DB_*`, `EMPLOYEE_DIRECTORY_TABLE`, `EMPLOYEE_DIRECTORY_*_COLUMN`, `EMPLOYEE_DIRECTORY_DOMAINS`
Snipe-IT	`SNIPEIT_URL` (the API address, ending in `/api/v1`), `SNIPEIT_TOKEN`, `SNIPEIT_CACHE_MINUTES` (5 by default). Without `SNIPEIT_URL` its pages say that Snipe-IT is not set up
Sessions and web	`SESSION_DRIVER=file`, `SESSION_LIFETIME`, `SESSION_IDLE_TIMEOUT`, `SESSION_ABSOLUTE_TIMEOUT`, `SESSION_SECURE_COOKIE`, `TRUSTED_PROXIES`
API	`SANCTUM_TOKEN_PREFIX`, `CORS_ALLOWED_ORIGINS`

Updating production

1. `git pull` on the server, then read that version's **Deploy** line in `CHANGELOG.md`.
2. `docker compose up -d --build`.
3. If the version brings migrations: `docker compose exec app php artisan migrate`. Migrations never run automatically, because the database is shared; once per database is enough.
4. If there are new settings: add them to the server's `.env`, then `docker compose restart app`.
5. If a migration adds a menu and is run from elsewhere (no shared cache): `docker compose exec app php artisan cache:forget rbac_menu_route_key_map`, so that the map of routes to menus is rebuilt.

Migrations after 1.10.0: `add_locale_to_users` (1.13.0) and `add_snipe_it_menu` (1.15.0). The health check with a token names the migrations that have not run yet.

The image only holds what `.dockerignore` lists (a list of what may go in): the code (`app`, `bootstrap`, `config`, `database`, `lang`, `modules`, `public`, `resources`, `routes`), assets, `VERSION`, `CHANGELOG.md` and `CHANGELOG.en.md` (for the Version history), without `.env`, Excel/Word/PDF data, tests, and the `docs/` folder. The `lang/` folder only joined that list in 1.21.3: from 1.13.0 to 1.21.2 the image carried no shared texts, so production showed keys such as `login.title`. Since then `DockerImageTest` makes sure that every folder the application reads, including Composer autoload folders such as `modules`, is on the list.

Health check

Address	For	Content
`GET /up`	The container healthcheck, uptime monitors	The application answers (`{"status":"up"}`), without the database, a session or a token. A restart does not repair a database, so the database is not checked here
`GET /api/v1/health` without a token	Monitors: does the website work normally	Four essential checks: `web` (the login page is opened as a browser would, through routing, the session, the language and the template, and must show its form; without the account and session of that request, and the page's session is deleted again), `database` (it answers, and the `users`, `menu` and `radios` tables can be read), `storage`, `session`. One failing means `down` (HTTP 503)
`GET /api/v1/health` with an API token	The full check	The essential checks plus `cache`, `migrations` (those not run yet), `snipe_it`, `microsoft` (the application can sign in to Microsoft), `ldap`, `hr_database`. Each check is `ok`, `warn`, `fail` or `skipped` (not set up here), with its time and reason; the overall status is `degraded` when a supporting service has a problem

10. Testing

Layer	Content
PHPUnit	663 tests in 67 files at version 1.21.0: register rules, import, service, Needs Attention, stock opname, menus, API, health check, sign-in and sessions, access, Audit Log, SharePoint (a simulated Microsoft), Snipe-IT (a simulated Snipe-IT), notifications, languages (both have the same keys and placeholders), versions and the version history, the contents of the Docker image (`DockerImageTest`, since 1.21.3). Always on a LocalDB database whose name ends in "Test"; the suite refuses any other database
Page scripts	27 Node checkers (`tests/tools/*.mjs`) run the pages' inline JavaScript without a browser, called from PHPUnit, some of them also on English pages; plus a syntax check of every script in the Blade views
Browser	Checks in headless Edge on a copy of the application with a sample database and made-up data, without connections to Microsoft, SharePoint, Active Directory, HR or Snipe-IT (a local simulated Snipe-IT), at widths of 390, 1366, 1440 and 1920 px

Tests never reach outside systems: `phpunit.xml` empties the Microsoft, SharePoint, LDAP, HR and Snipe-IT settings, and an HTTP request that is not faked makes the test fail.

11. Versions and releases

- **Semantic Versioning** since 1.5.0. The version number is in the `VERSION` file and shows in the application's footer and in the API documentation.
- Every change pushed to `main` is one release: `VERSION` goes up, its part is added to `CHANGELOG.md` (in Indonesian, with a Deploy line) and in English to `CHANGELOG.en.md` (without a Deploy line), and an annotated git tag `vX.Y.Z`. Changes that only touch documentation or tests do not raise the version.
- **Version history**: the version number in the footer opens a dialog read from `CHANGELOG.md`, or from `CHANGELOG.en.md` when the page is in English (`Changelog`, `ChangelogController`; a version not yet in the English file keeps the Indonesian notes). It is a timeline: each series is one node, its patch releases branch off below it, and every version takes the colour of its strongest kind of change. Section headings must be one of the Keep a Changelog kinds; `ChangelogDialogTest` refuses other headings, and versions that are in one file only or whose kinds of change differ between the two files. The legend and its kind labels (Minor release, Patch release, Added, Changed, …) are always in English.
- MAJOR for changes that no longer fit what came before, MINOR for new features, PATCH for fixes.
- Throughout version 1.x the API stays `/api/v1` and only grows.
- Going back to an earlier version: `git checkout vX.Y.Z` and rebuild; tables and columns from later versions stay in the database, and menus whose pages do not exist in the old version show without a link.
- Large work is done on a separate branch and released once when it is merged: the new look and the phase 6 abilities on the `design-refresh` branch (its plan and decisions in `docs/PLAN.md`) became the MINOR release 1.10.0, and the language choice on the `i18n` branch (its rules in `docs/I18N.md`) became 1.13.0.

12. Design decisions and open points

Decision	Reason
One row = one physical radio (SN)	The old register mixed ID slots and radios; the SN is the physical identity that stays
Status split into Location and Condition	One Status column mixed where the radio is with the state it is in
Contradicting Excel data is marked, not guessed	Guessing hides problems; the Needs checking mark makes them checkable
A Radio ID with F in its MDC Code is refused	F is a wildcard in MDC-1200
A radio that leaves for good is archived, not deleted	The history stays whole and the Radio ID can be used again
Rules in services, used by the pages and the API	One place for the rules, so the API cannot skip the rules of the pages
Workbook cell changes through a request and a second person's approval	The workbook is shared; writes by the application identity are recorded under the application's name in SharePoint, so the person's trail is kept in the application
No scheduler	Time-based notifications can simply be made when the person opens the application
File sessions with 30-minute and 8-hour limits	Following OWASP and NIST; cookie sessions failed to carry messages larger than 4 KB
No front-end build	The Tabler theme is ready-made; maintenance only needs Blade and jQuery
A design layer on top of Tabler (`radio-ui.css`)	A new look without replacing the theme and without a build: colours become `--rm-*` tokens, components become classes every page reuses
Service, check and opname tables without a foreign key to radios	An import with "delete the old data" deletes every radio; the copied labels keep old tickets and sessions readable. Such an import is refused while a ticket or session is open
Direct row edits only for approvers	Whoever may approve others need not wait for an approval of their own; the trail stays in the requests marked `direct` and in the Audit Log
The camera through `BarcodeDetector`	No extra library; browsers without it use typing or a handheld scanner
The handover signature is for printing only	The recipient signs the printed paper; the application need not keep the signatures of people outside the ICT team
Login page, sidebar and some dialog sizes follow production 1.9	Users are used to the old layout (measured element by element in production); colours and shapes stay from the new design
Page title in the top bar, the path below it, no subtitles	The application owner's choice: the title does not show twice and the content moves up
Indonesian and English, data stays in Indonesian	Everyone chooses their language, but data that is stored and used by others (history, Audit Log, Excel, API labels) does not change with the reader's language
Kinds of release and change always in English	The application owner's choice: the Semantic Versioning and Keep a Changelog terms are used as they are, in both languages

Decision	Reason
Snipe-IT is only read	Snipe-IT stays the source of IT asset data. The application has no route that changes anything there, so even a token with too wide access cannot be used to change data through this application
The Docker image from a list of what may go in	`.dockerignore` names what goes in, not what is refused, so secrets and real data cannot slip in by accident. The flip side is that a new folder can be left out: `lang/` was missing from 1.13.0 to 1.21.2, so since 1.21.3 a test checks it
Health check in two levels	`/up` without the database for Docker, because a restart does not repair a database; `/api/v1/health` checks down to the database, and its details are only for token holders, without server names
Template leftovers removed in stages	Version 1.20.1 removed what was proven unused. The ApprovalManagement module, the registration and password reset controllers (their routes are off), and `package.json` and Vite are deliberately still there

Open points

- Writing to the real SharePoint workbook (an approved request or a direct row edit) has never been tried; try it first on an unimportant cell.
- The SharePoint write permission should be narrowed from `Sites.ReadWrite.All` to `Sites.Selected` for the one ICT site.
- The Snipe-IT token should come from a Snipe-IT account that may only view.
- The components and files of a Snipe-IT asset cannot be shown yet, because the Snipe-IT 6.2 API does not provide them; the details page links to Snipe-IT.
- The template leftovers still present (ApprovalManagement, the registration and password reset controllers, `package.json` and Vite) can be removed once they are certainly unused.
- There are no e-mail notifications yet (the application's mail setting is still `log`).
- Barcode/QR labels for radios do not exist yet; the Stock Opname camera cannot be used on an iPhone without a scanning library.
- Signing with a Wacom pen has not been tested on a real device.
- Local password sign-in should be closed once every account uses Microsoft or LDAP.